

Práctica 2: Listas en Python

De la implementación del TAD a resolver problemas con list

AED I — Bloque I

2 h en clase + 1 h en casa

Algoritmos y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

Tres ideas antes de empezar

1. `list` en Python se comporta conceptualmente como un array dinámico: podemos acceder directamente por índice.
2. Que una operación sea cómoda de escribir no significa que sea barata: insertar o borrar al principio obliga a mover elementos.
3. En los ejercicios importa tanto que el resultado sea correcto como poder explicar la complejidad de la solución.

Operaciones básicas de list

Operación	Python	Coste
Añadir al final	<code>lista.append(x)</code>	$O(1)$ amortizado
Acceder por posición	<code>lista[i]</code>	$O(1)$
Buscar un elemento	<code>x in lista</code> / <code>index(x)</code>	$O(n)$
Insertar en posición i	<code>lista.insert(i, x)</code>	$O(n)$
Eliminar en posición i (posición -1 por defecto)	<code>lista.pop(i)</code>	$O(n)$
Tamaño	<code>len(lista)</code>	$O(1)$

Pregunta: ¿por qué `lista.pop()` es mucho más barato que `lista.pop(0)`?

Reglas de los ejercicios

- Trabajaremos únicamente con list de Python.
- No usar sort(), sorted() ni reverse() cuando el ejercicio pida implementar el algoritmo.
- Evita crear estructuras auxiliares salvo que el enunciado lo permita.
- Al terminar, indica la complejidad temporal de tu solución.

Ejercicio 1 - Ordenación por inserción

Implementa ordenación por inserción sobre una list, modificándola in-place.

```
def insertion_sort(lista):  
    # completa  
    pass  
  
datos = [7, 3, 5, 2, 9, 1]  
insertion_sort(datos)  
assert datos == [1, 2, 3, 5, 7, 9]
```

- Usa índices y asignaciones.
- No uses `sort()` ni `sorted()`.
- Prueba lista vacía, un elemento y repetidos.
- ¿Mejor caso? ¿Peor caso?

Ejercicio 1 - Traza antes de programar

Lista inicial: [7, 3, 5, 2]

i = 1 **valor = 3** → [3, 7, 5, 2]

i = 2 **valor = 5** → [3, 5, 7, 2]

i = 3 **valor = 2** → [2, 3, 5, 7]

- ¿Qué elementos se desplazan en cada paso?
- ¿Cuántas comparaciones hay si la lista ya está ordenada?
- ¿Y si viene en orden inverso?

Ejercicio 2 - Fusionar dos listas ordenadas

Dadas dos listas ya ordenadas, construye una tercera lista ordenada en una sola pasada.

```
def fusionar(a, b):  
    # completa  
    pass  
  
assert fusionar([1, 4, 7], [2, 3, 9]) == [1, 2, 3, 4, 7, 9]  
assert fusionar([], [2, 5]) == [2, 5]
```

- Usa dos índices: i y j .
- No concatenes y ordenes después.
- Añade al resultado con `append()`.
- Objetivo: $O(n + m)$.

Ejercicio 3 - Eliminar duplicados conservando el orden

Crea una nueva lista con la primera aparición de cada valor, manteniendo el orden original.

```
def sin_duplicados(lista):  
    # no uses set  
    pass  
  
assert sin_duplicados([3, 1, 3, 2, 1, 4]) == [3, 1, 2, 4]  
assert sin_duplicados([]) == []
```

- Solo puedes usar listas.
- Analiza el peor caso.
- ¿Por qué puede acabar siendo $O(n^2)$?

¡A programar!

Siguiente práctica: Pilas y Colas